Global cut surgery for tensor-network contraction ordering

Jin-Guo Liu[1⋆]

1 Advanced Materials Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

⋆ jinguoliu@hkust-gz.edu.cn

Abstract

Simulated annealing over contraction trees is the strongest general-purpose optimizer of tensor-network contraction orders, yet late in every run its dominant contraction—the tree's *waist*—freezes. Across 835 instrumented converged states, this frozen waist is *never* a locally minimal balanced cut: a bounded Fiduccia–Mattheyses pass finds a strictly cheaper cut every time, by 3–12 bits. *Global cut surgery* exploits the blind spot: extract the waist, improve it as a graph cut, cold-rebuild both sides, accept on measured cost. It sets three records on ten benchmarks including the Sycamore circuit, holds the high-quality wall of the time–quality Pareto front, and gains with scale; certified bounds and twenty falsified alternatives show search control alone cannot win.

Received Date
Accepted Date
Published Date

Contents

1 Introduction

The cost of contracting a tensor network is governed by the order in which its tensors are combined. Finding a good order is the computational bottleneck of classical random-circuit simulation [1, 2], of quantum error-correction decoding, and of exact probabilistic inference, and a rich ecosystem of optimizers has grown around the problem: simulated-annealing tree refiners [3], hyper-optimized portfolios over greedy and hypergraph-partitioning constructions [4, 5], treewidth machinery from the parameterized-algorithms community [6, 7], exact methods for small networks [8], and, recently, learned optimizers—reinforcement-learning policies [9] and LLM-evolved heuristics [10]. Among these, tuned simulated annealing over contraction trees is in practice the strongest general-purpose refiner: as we show below, thirteen mechanically distinct search paradigms and an independent toolchain all converge onto its frontier, and on small instances that frontier is *certifiably* locally optimal. Improving on it therefore requires attacking a structural weakness, not a better schedule.

This paper identifies such a weakness and builds the algorithm that exploits it. The weakness is the annealer's *waist*: with the total cost a log-sum-exp over all contractions, the converged tree's cost is dominated by its single most expensive contraction, which induces a bipartition of all tensors. Local tree rewrites move one subtree at a time and can cross between distinct good bipartitions of the same size only through long, individually uphill move sequences, so late in the schedule the waist freezes. Our motivating measurement (Sec. 4) is that this frozen cut is not merely hard to improve locally—*it is never a locally minimal balanced cut of the tensor graph at all*. Across 835 instrumented calls on converged states of real benchmark networks, a bounded Fiduccia–Mattheyses pass found a strictly cheaper cut of comparable balance every single time.

Global cut surgery (Sec. 5) turns that observation into an optimizer: extract the waist bipartition from the incumbent tree; improve it as a pure graph-cut problem, ignoring the tree; if a cheaper comparable-balance cut exists, promote it to the tree top, cold-rebuild each side with fixed-work sub-anneals, splice, and accept only if the independently rescored total cost drops; iterate on the (possibly moved) waist. An exact rank-non-increasing simplification pass—which removes up to 89% of the tensors of real-provenance networks before any search begins—runs as preprocessing.

We evaluate the algorithm under a hardened protocol (matched single-threaded budgets, independent rescoring of every tree from its topology, per-run relabeling, confirmation-based records; Sec. 2) and re-measure every headline comparison on a single dedicated machine against the reference Julia implementation, its published benchmark suite, and cotengra (Sec. 6). Surgery sets records on three of ten instances, its run distribution separates fully from tuned annealing's on surface code d=21, it holds the entire high-quality wall of the time–quality

Pareto front, and its advantage grows with surface-code distance. Its boundaries are reported with the same care: random 3-regular expanders yield the advantage back at long budgets, and two instances repel it entirely.

Three further results frame the algorithm's necessity and scope. First, the incumbent must be tuned before being beaten: a single shipped default (an aspirational memory target) costs 20×–2100×, more than any mechanism difference we measured (Sec. 3). Second, on small converged instances nothing can win: we certify the annealing frontier width-optimal, localize the optimum to $[53, 61.5]$ on the circuit proxy, and prove that bounds computable from the isoperimetric profile can go no further (Appendix A)—which is precisely why the gains must come from changing the *input* rather than the search. Twenty falsified alternative mechanisms sharpen the same lesson (Appendix B). Third, on probabilistic-inference workloads the method's scope splits by structure: annealing-family methods, ours leading, win sparse pedigree networks, while elimination orderings win dense Bayesian networks, and the shipped defaults of TensorInference.jl sit 12–30 bits above the frontier either way (Sec. 7).

2 Setup and evaluation protocol

Instances and cost model. A tensor network is a hypergraph $G = (V, E)$: vertices are tensors and (hyper)edges are indices; contracting to a scalar (or a small open-index set) requires a full binary contraction tree over the tensors. Writing cost_v for the $\log_2$ floating-point cost of internal node v (the sum of $\log_2$ dimensions over the union of the operands' indices), the two standard quality measures are the total time cost

$$\text{tc} = \log_2 \sum_v 2^{\text{cost}_v}, \tag{1}$$

a log-sum-exp over all $|V|-1$ contractions, and the space cost sc, the $\log_2$ size of the largest intermediate tensor. Minimizing (1) is NP-hard [11], a hardness discovered independently three times: for join ordering in relational databases [12], for sum-of-products loop optimization in compilers [13], and for tensor networks—and algorithms migrate across these communities, most recently join-ordering algorithms into tensor networks [14]. All numbers in this paper are $\log_2$ quantities; a difference of 1.0 is a factor of two in flops. Equation (1) is the root of everything that follows: because tc is a log-sum-exp, the converged tree's cost is carried almost entirely by its most expensive contraction—the *waist*.

Table 1 lists the ten benchmark instances. Two are closed synthetic networks used for the certification study (a 250-tensor random 3-regular graph and a 561-tensor Sycamore-*like* circuit proxy, which we distinguish explicitly from the real circuit). The remainder are imported from the OMEinsumContractionOrders benchmark suite [15] with provenance recorded at import: the real Sycamore 53-qubit, 20-cycle network, surface-code syndrome networks at distance $d = 9 \ldots 21$, an independent-set counting network on a king graph, a 1000-vertex random 3-regular graph, a 97-qubit random-circuit network, the $n = 28$ queens counting problem, a deep-belief-network inference instance, and a 27-qubit quantum Fourier transform. Section 7 adds ten hard instances from the UAI-2014 inference competition, exported directly from the artifact shipped with TensorInference.jl.

Protocol. Every optimizer run gets 90 seconds of single-threaded wall clock per instance unless stated otherwise. Emitted trees are rescored by an independent checker that recomputes tc and sc from the tree topology and the original index sets alone; optimizer-reported numbers are never trusted. Instances are randomly relabeled per run, so memorized answers keyed on input encoding do not transfer. A best-known value ("record") only moves when a run

instance	tensors	indices	provenance
reg3_250	250	372	synthetic, closed (certification study)
sycamore_m20	561	963	synthetic Sycamore-like proxy (certification)
sycamore_53_20_0	3369	—	real Sycamore 53q, m=20 [4]
surfacecode_d9–d21	219–2203	—	surface-code syndrome networks
ksg	5197	—	king-graph independent-set counting
reg3_1000	1000	1500	random 3-regular, n=1000
rqc_97_m24	1238	—	97-qubit random circuit
nqueens_28	4252	—	28-queens counting
dbn_13	572	44 labels	deep-belief-network inference (UAI DBN_13)
qft_27	405	—	27-qubit QFT, 27 open indices

Table 1: Benchmark instances. The first two are the closed synthetic networks of the certification study (Appendix A); the rest are imported real-provenance benchmark networks. Ten additional UAI-2014 inference instances are introduced in Sec. 7.

beats the incumbent by more than 0.05 and a second run under fresh relabeling confirms; the worse of the two runs is recorded. On instances above 1000 tensors, where single-run variance exceeds method differences, scored values are medians of three runs. Per-child CPU-time accounting rejects hidden parallelism. Reference rows (the tuned annealer of Sec. 3, and cotengra's hyper-optimizer and SA refiner [4]) are measured best-of-three with the identical pipeline.

Same-machine re-measurement. Because the record board accumulated on a development machine, all headline comparisons were re-measured from scratch on a single dedicated 2-core Linux server: 305 scored runs covering budget scaling (90/300/900 s, three repetitions), run distributions (15 repetitions), and the surface-code family (five repetitions), plus a matched-budget ladder of the reference Julia implementation (OMEinsumContractionOrders.jl: TreeSA at both memory-target settings and one/four/eight restarts, HyperND, and three treewidth back-ends) run through that benchmark suite's own harness, and the inference batch of Sec. 7. Every cross-method statement in Secs. 6 and 7 is same-machine and matched-budget.

3 Tuning the incumbent: a prerequisite

Beating an untuned incumbent is not evidence of algorithmic progress, and the incumbent here ships untuned. The TreeSA-family annealers [3] optimize a composite objective in which the space cost is pulled toward a target $\mathrm{sc}_{\mathrm{target}}$, with a penalty active whenever $\mathrm{sc} > \mathrm{sc}_{\mathrm{target}}$. The implementation we study (omeco, a Rust re-implementation aligned with OMEinsumContractionOrders.jl) inherited the shipped default $\mathrm{sc}_{\mathrm{target}} = 20$. On instances whose good trees have naturally larger intermediates—the optimum-quality trees on our two closed instances have sc = 34 and 53—this default is not merely unhelpful. It actively drags the search into a low-memory, high-flop region of tree space.

Figure 11 (Appendix C) shows the resulting tc at a fixed 90-second budget as a function of $\mathrm{sc}_{\mathrm{target}}$. The transition it reveals is a cliff, not a slope: every target below the natural space cost of good trees is roughly equally harmful (medians 45.6–47.0 on reg3_250, 71–77 on sycamore_m20), quality snaps to the frontier exactly at the natural value ($\mathrm{sc}_{\mathrm{target}} = 34$ and 53), and stays there unchanged through $\mathrm{sc}_{\mathrm{target}} = \infty$, i.e., with the memory objective deleted. The harm is strictly one-directional. At the shipped default the damage is 4.3 $\log_2$ units on reg3_250 and 9.7 on sycamore_m20 (20× and 800×); on the real Sycamore net-

work it reaches 11.1 units (2100×, Sec. 6). The cliff is visible in *published* data as well: the benchmark suite of OMEinsumContractionOrders.jl reports Julia TreeSA at tc = 71.0 on the real Sycamore network with its shipped $\mathrm{sc}_{\mathrm{target}} = 20$, within 1.6 of our re-measurement of the same configuration, and 11 units above the same annealer with the target removed.

The practical rule is: *never anneal toward a memory target below the natural space cost of good trees; absent a physical memory budget, use the natural value or no target at all.* Every comparison in the remainder of this paper is against the annealer with this fix applied—the *tuned reference*.

Why not better search control. Before targeting the waist, we verified that no search-control change can win where the tuned annealer converges: thirteen mechanically distinct search paradigms and an independent toolchain (cotengra) all land within 0.35 of its frontier on the two closed instances, the frontier is certifiably width-optimal with the optimum localized to $[53, 61.544]$ on the circuit proxy, and we prove that every lower bound computable from the isoperimetric profile is capped below the balanced-cut value. The full certification — the thirteen-mechanism convergence, two sum-form bound theorems, and the impossibility result — is given in Appendix A, and twenty further falsified mechanisms are taxonomized in Appendix B. The lesson they encode is uniform: global information helps only when injected as *input*, never as search control. The remaining question is where such an input exists, and the answer is the waist.

4 The frozen waist is never a minimal cut

Every converged annealed tree has a waist: by (1) its cost is dominated by the argmax-cost contraction, whose leaf set A (against complement B) is a bipartition of all tensors, and whose cost is essentially the weight of the cut (A, B) in the tensor hypergraph. Local tree moves—the associativity rewrites of the annealer [3]—transfer one subtree across this bipartition at a time; crossing to a *different* good bipartition of comparable balance requires a long sequence of individually uphill steps, which a cooled annealer will never take; Fig. 1 draws the resulting landscape picture. We also know the freeze is not repairable by small exact windows: exhaustive splicing of all subtrees up to 16 leaves leaves frontier trees unchanged (they are window-exact optimal, Appendix B), while the dominant contraction spans a constant fraction of the network.

The question is whether the frozen waist is actually good *as a cut*. The right tool for that question is Fiduccia–Mattheyses (FM) refinement [16], the classic linear-time partitioning heuristic from circuit design; since it is the one ingredient this paper imports from outside the tensor-network toolbox, we state it in full. Given a bipartition (A, B) of the tensors, assign every tensor a *gain*: the decrease in the cut weight $w(A, B)$—here the summed $\log_2$ dimensions of the indices straddling the cut—if that tensor alone switched sides. A pass repeatedly moves a highest-gain tensor across the cut *even when that best gain is negative*, locks it, updates the gains of its neighbors, and rejects moves that unbalance the two sides beyond a tolerance; when all tensors are locked, the pass rolls back to the best prefix of its move sequence. The tolerated uphill moves before rollback let a pass carry the boundary across ridges that no sequence of individually improving single moves can cross—the same barriers that freeze the annealer—and a bucket structure over gains makes a full pass linear in the network size. *Bounded* FM caps the number of passes and move attempts (scaling with instance size, nothing tuned per instance), so a call costs a fixed, small slice of the budget.

We instrumented every surgery call of the algorithm of Sec. 5 on its two record runs: at each converged incumbent, extract the waist bipartition and run bounded FM for a cheaper

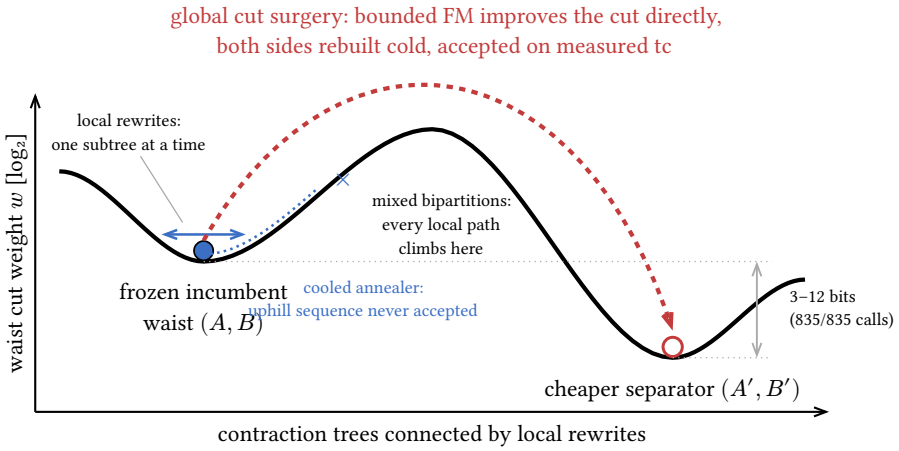

Figure 1: Why the annealer freezes and surgery does not. The landscape is the waist cut weight w over contraction trees, with neighboring trees connected by the annealer's local rewrites; each rewrite moves one subtree across the waist bipartition, so a distinct separator (A', B') of comparable balance is reachable only through a long sequence of mixed bipartitions of higher cut weight, which a late-schedule annealer never accepts (dotted attempt). Surgery side-steps the barrier by improving the cut in cut space—bounded FM, seeded to jump separators (Sec. 4)—and rebuilding both sides (Algorithm 1, lines 5–8). The well-depth gap is not hypothetical: it is 3–12 bits on every one of the 835 instrumented calls (Fig. 2a). Schematic.

cut of comparable balance on the tensor hypergraph, recording the incumbent's cut weight, the best alternative found, and the structure of the improved cut.

Figure 2a shows all 835 pooled calls on two instances, and **this panel is the paper's motivating result**: every single point lies strictly below the diagonal. *The annealer's waist is never a locally minimal balanced cut*—bounded FM finds a strictly cheaper cut of comparable balance on every call, by margins of 3–12 bits. The improved cuts are sparse rather than near-clique (clique fraction 0.01–0.05): genuine alternative separators, not degenerate rearrangements of the same one. The blind spot is structural and persistent: it does not shrink as the incumbent improves, and it recurs at the new waist after every accepted repair. An optimizer that can see cuts globally therefore always has a move available exactly where the annealer is most stuck. The claim is scoped to the instance families measured here: Sec. 7 exhibits a hub-dominated family on which the balanced waist *is* sometimes locally minimal, and maps why.

5 The algorithm: global cut surgery

Global cut surgery operationalizes the observation of Sec. 4; Algorithm 1 states the loop, and Fig. 3 illustrates both of the pipeline's mechanisms on a toy network. The optimizer maintains an incumbent contraction tree produced by a standard annealing engine, and interleaves, at every outer iteration, a surgery step that treats the waist as a graph problem:

1. **Extract** (Algorithm 1, lines 3–4). Locate the argmax-cost node of the incumbent (costs recomputed exactly from the index dimensions); its leaf set A versus complement B is the waist bipartition.

2. **Improve the cut, not the tree** (line 5). Run the bounded FM passes of Sec. 4 on the tensor hypergraph, with gain defined as the exact reduction in summed $\log_2$ dimensions

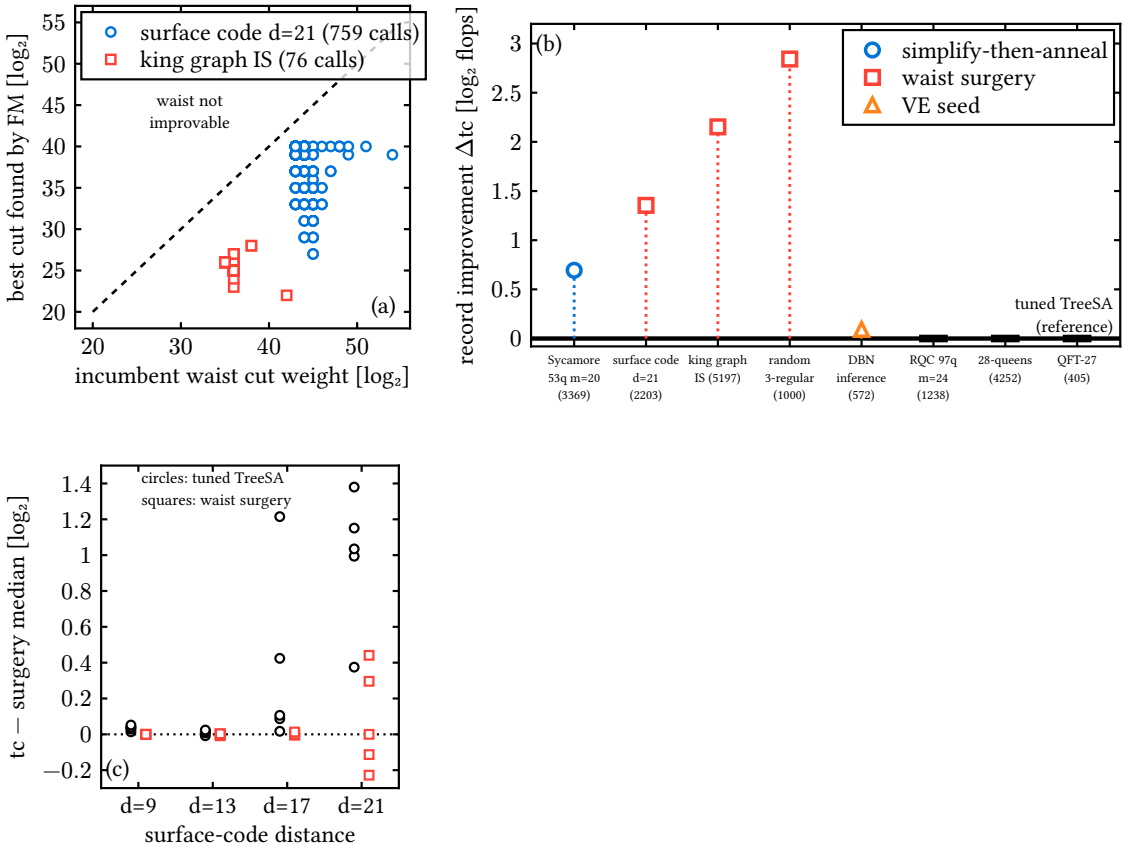

Figure 2: The blind spot and the advantage it buys. (a) Every surgery call on the two record runs: incumbent waist cut weight versus the best comparable-balance cut found by bounded FM. All 835 points lie strictly below the diagonal—the annealed waist is never locally minimal (Sec. 4). (b) Record board: confirmed improvement over the tuned TreeSA reference on each instance (validator protocol: confirm-twice, worse-of-two, per-run relabeling); marker shape encodes the mechanism, thick dashes mark instances where the tuned reference remains the record holder. (c) Surface-code family: all repetitions at 90 s versus code distance, anchored at the surgery median per distance—the surgery advantage grows with d while its run-to-run spread stays collapsed.

of the straddling labels (open output labels are excluded—they are invariant), and balance constrained to $|A| \pm 18\%$. The search is seeded from the incumbent cut plus four boundary-BFS alternative bipartitions (one from the highest-degree tensor, three from random starts), so it can jump to a different separator rather than only polish the current one.

3. **Rebuild cold, with fixed work** (lines 7–8). If a strictly cheaper cut of comparable balance was found, promote it to the tree top: each side becomes an independent, much smaller ordering problem (its open indices are the new cut plus its external labels, computed by outside-occurrence counting so the rebuilt tree's top node matches the independent scorer exactly). Anneal each side from scratch with a *fixed-work* budget (three V-cycles, linear schedules), join, and splice into the incumbent.

4. **Accept on measurement** (lines 9–10). Adopt the rebuilt tree only if its independently recomputed total cost improves; then iterate—the waist typically moves, and surgery

Algorithm 1 Global cut surgery. $\mathcal{A}$ is a standard annealing engine (tuned per Sec. 3); $w(A,B)$ is the summed $\log_2$ dimension of the labels straddling the bipartition (A,B), excluding open output labels. All costs are recomputed exactly from the index dimensions; nothing is tuned per instance.

Require: tensor network G (after the exact simplification pass of Sec. 5.1), time budget
$T \leftarrow \mathcal{A}(G)$ ▷ incumbent contraction tree
while budget remains **do**
  $v^* \leftarrow \arg\max_v \mathrm{cost}_v(T)$ ▷ the waist: argmax-cost contraction
  $(A,B) \leftarrow$ leaf bipartition of T induced by v^*
  $(A',B') \leftarrow \textsc{BoundedFM}\big(G,(A,B)\big)$ ▷ gain $= w$ reduction; balance $|A| \pm 18\%$; seeded from (A,B) plus four boundary-BFS bipartitions
  if $w(A',B') < w(A,B)$ **then**
    $T_L \leftarrow \mathcal{A}(G[A'])$, $T_R \leftarrow \mathcal{A}(G[B'])$ ▷ cold rebuilds, *fixed work* (three V-cycles); open indices by outside-occurrence counting
    $T' \leftarrow \textsc{Splice}(T_L, T_R)$
    if $\mathrm{tc}(T') < \mathrm{tc}(T)$ **by independent rescoring then**
      $T \leftarrow T'$ ▷ waist moves; next pass repairs the new waist
    end if
  else
    run annealing sweeps of $\mathcal{A}$ on T ▷ interleaved local refinement
  end if
end while
return T ▷ emitted eagerly throughout (anytime)

repeats at the new waist until no improving cut is found or the budget ends.

Two design choices matter beyond the loop itself. *Fixed work, not fixed time*: giving each rebuild a deterministic amount of annealing work (rather than a wall-clock slice) collapses run-to-run spread from 0.8 to 0.08 bits—the algorithm's variance behavior is set by the rebuild policy, not by the machine. *Surgery needs its base engine, and vice versa*: an ablation that keeps only the ratchet base engine plateaus far above the full algorithm (surface-code d=21: 49.6 ratchet-only versus 47.38 with surgery; king graph: 50.0 versus 36.8), while surgery without a competent annealed incumbent has nothing worth repairing. All knobs scale with instance size; nothing is tuned per instance.

Relation to prior refiners. Refinement layered on top of a construction is an active line: the TreeSA family itself [3], simulated annealing over partition assignments [17], and, closest to this work, nearest-neighbor-interchange refinement of hyper-optimizer output, whose gain over cotengra grows with bond dimension [18]. All of these move the *tree* locally. Surgery differs in kind: it improves the dominant *cut* as a graph problem and hands the result back as input to a rebuild—and the measurement of Sec. 4 is precisely that the local refiners' fixed point leaves this cut improvable every time.

5.1 Preprocessing: exact simplification

Real-provenance networks carry large amounts of *rank-non-increasing* structure: tensors whose contraction with a neighbor can never increase any intermediate rank (rank-1 spiders, dangling legs, series chains). Preprocessing of this kind is established practice (cuTensorNet's pathfinder simplifies before partitioning [19], and opt_einsum removes single-tensor reductions [20]); our pass is the exact rank-non-increasing closure with splice-back, so it composes with the record protocol. Before any search, the pass contracts this structure away greedily;

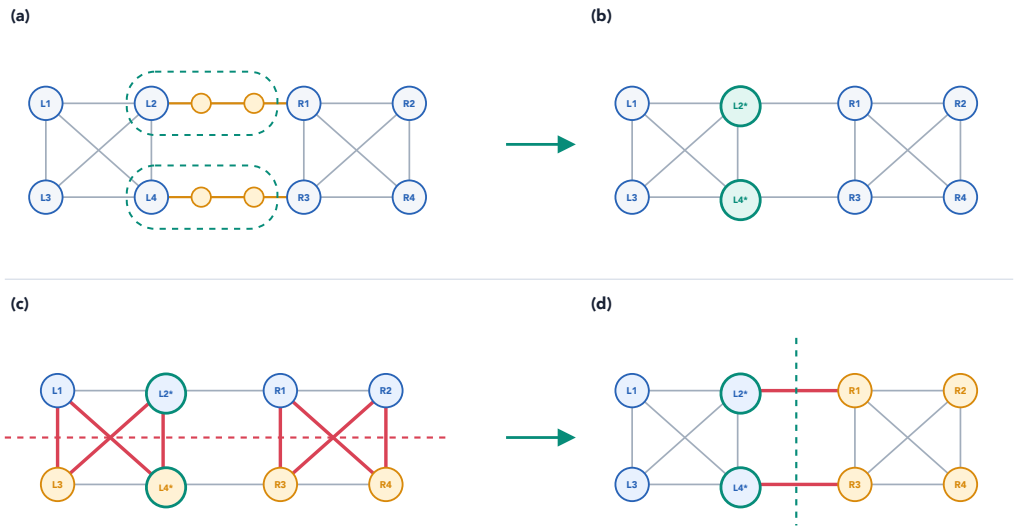

Figure 3: The two mechanisms on a toy network: two dense clusters (each a K_4) joined by two sparse bridges. (a,b) Preprocessing: the chain tensors on the bridges are rank-non-increasing, so the exact simplification pass absorbs them ($L_2, L_4 \to L_2^*, L_4^*$) before any search begins. (c,d) Surgery: a converged annealed tree can freeze with a poor waist—the horizontal bipartition (c), which cuts through both clusters—while the cheap comparable-balance cut through the two bridges (d) is invisible to local tree moves; bounded FM on the tensor graph finds it, and both sides are rebuilt cold and spliced.

the optimizer runs on the reduced network, and the contracted fragments are spliced back into the final tree as fixed subtrees. The pass is exact and costs milliseconds.

Its effect is largest exactly where synthetic benchmarks never look (Fig. 12a, Appendix C): 88.7% of the real Sycamore network's 3369 tensors are removed before annealing begins (3369 → 381), 65.5% of surface-code d=21, 33.9% of nqueens_28—and 0% of the random 3-regular control, which is why the identical mechanism, correctly measured as useless on the pre-simplified synthetic proxy two cycles earlier, was nearly discarded. A matched-budget ablation on the real Sycamore network isolates the contribution: the same anneal reaches tc = 61.3 with simplification and 76.0 without it, a $2^{14.7}$ ($\sim$27000×) difference from preprocessing alone. Figure 12b shows the resulting record march: three successive simplify-then-anneal implementations drove the Sycamore record 60.605 → 60.169 → 60.104 → 59.911.

5.2 Seeding and anytime behavior

The pipeline emits its incumbent eagerly (atomic writes), so it is an anytime optimizer, and its construction stage is pluggable. Two components earn their place. *Elimination seeds*: on hypergraph-like instances with few, high-degree labels, seeding the annealer with a min-fill variable-elimination order gives it a start (tc = 28.8 at 0.2 s on the DBN instance) that greedy construction (44.7) cannot approach. During the campaign this mechanism appeared to have a sharp boundary—at 4086 labels the seed seemed to freeze—but the freeze later proved to be largely an implementation artifact of the greedy construction stage, and a quotient-graph reimplementation removes it entirely (Sec. 7, "a seed that scales"). *Racing*: arms with budget-fraction linear schedules, composed by a router, produced three records in a single run during development; a probe-leader policy never won. Harness-clocked time-to-frontier records exist as well (16.1 s to the reg3_250 frontier versus 20.2 s for the reference; 0.2 s on

the DBN instance).

6 Evaluation

Records. Figure 2b assembles the campaign's confirmed records against the tuned reference on all ten instances. Global cut surgery holds three (king graph 36.96, surface code d=21 47.40, reg3_1000 131.07—the only record on an expander in the entire campaign), the simplify-then-anneal line holds the real-Sycamore record (59.91), and the elimination seed holds the DBN record (29.00). On four instances—both certified closed instances, the 97-qubit random circuit, and 28-queens—the tuned reference itself remains unbeaten, exactly as the certified null of Appendix A predicts for the converged cases. A representative surgery run (Fig. 10, Appendix C) shows the dynamics: each accepted rebuild drops the incumbent below the pre-surgery record, and the official median lands at 47.377.

The time–quality Pareto front. Figure 4 presents the same-machine campaign as a Pareto scatter, following the presentation of the OMEinsumContractionOrders benchmark study [15]: every optimizer configuration from every toolchain is one point, and the dashed line traces the non-dominated front. On both instances the front has the same anatomy: treewidth backends hold the fast/low-quality end (sub-second, 10–40 bits off best), HyperND the middle, and *global cut surgery holds the entire high-quality wall*—at every time tier (90/300/900 s) its points sit strictly left of every tuned-TreeSA point from either implementation. On the real Sycamore network surgery reaches 59.87 at 90 s while the tuned reference does not reach it even at 900 s (60.50)—a quality gap, not a speedup; on surface-code d=21, surgery at 90 s (47.39) matches the reference at 900 s (47.38)—a tenfold time-to-quality gain with both converging to the same floor.

Distributions and scaling. Figure 13 (Appendix C) gives the full run distributions. Over 15 repetitions at 90 s, the surgery and reference distributions on surface-code d=21 are fully separated—surgery's worst run (47.91) beats the reference's best (48.18); king-graph and reg3_1000 medians improve by 3.0 and 2.4; the elimination-seed mechanism shows a 14× tighter spread than the reference on the DBN instance. On the surface-code family the surgery advantage grows with distance (+0.04, +0.02, +0.11, +1.04 at $d = 9, 13, 17, 21$; Fig. 2c) while its run-to-run spread stays collapsed: global cut repair pays off precisely at scale, where the annealer's frozen waist is largest.

External baselines. Table 2 closes the loop against the reference Julia implementation, run on the same server through its own benchmark harness at a ladder of configurations, against the suite's published numbers, and against cotengra. Our records beat the best same-machine Julia configuration *at ten times our budget* on the three large structured instances (Sycamore 59.87 versus 61.35; surface code 47.40 versus 48.75; king graph 36.96 versus 38.27), and beat the entire published multi-optimizer suite by 6.8, 4.9, and 2.0 bits respectively. The cotengra column adds a protocol-matched cross-toolchain reference: on the small converged instances it confirms the shared frontier (reg3_250 40.5 versus our 39.9; the circuit proxy 62.4 versus 61.5), while on the large instances a single-threaded 90-second budget starves its sampling-based portfolio (69.0 on the king graph, 187 on the 97-qubit circuit), which quantifies how much of that toolchain's power lives in parallel trial throughput rather than in per-trial quality.

Where the advantage ends. Honest boundaries, all measured. On reg3_1000 the reference overtakes surgery at 900 s (129.8 versus 134.4): expanders have no cheap alternative sepa-

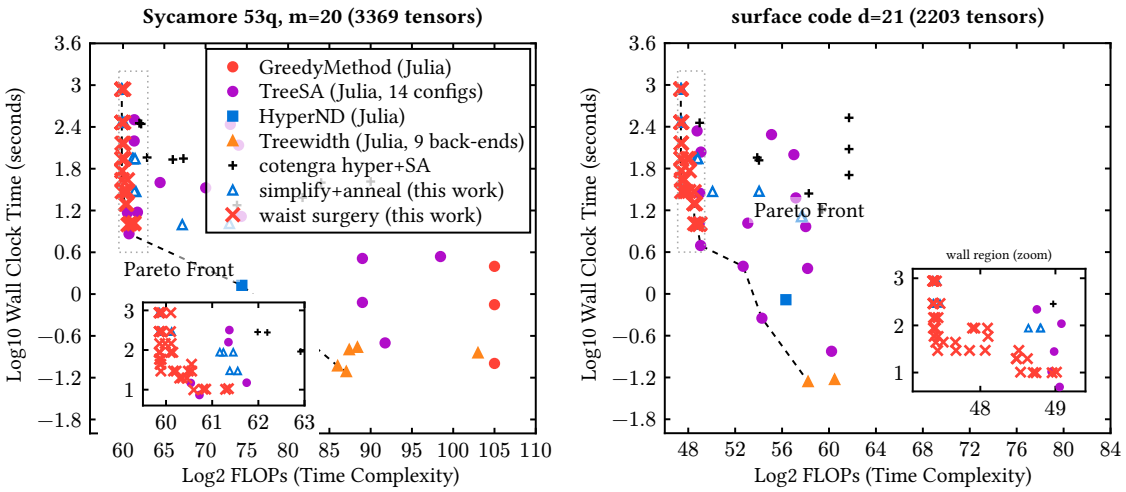

Figure 4: Time-versus-quality Pareto view of the same-machine campaign (dedicated 2-core server, all runs single-threaded), in the presentation style of the OMEinsumContractionOrders benchmark study [15]. Each point is one optimizer configuration or repetition. Baselines are the published implementations: Julia TreeSA across a 14-configuration ladder (both memory-target settings, 1–8 restarts, five sweep depths), GreedyMethod, HyperND, nine treewidth back-ends, and the cotengra hyper-optimizer with SA refinement at four budgets. Open blue triangles and red crosses are this work, at budgets from 10 to 900 s. Our tuned TreeSA port (not shown) coincides with the Julia TreeSA points wherever configurations overlap. The dashed line is the non-dominated front. Global cut surgery (crosses) holds the entire high-quality wall of the front on both instances; the insets zoom the wall region (dotted window), where the separation is explicit: on Sycamore the surgery front sits at tc = 59.85–59.9 from 60 s on (first reaching 59.89 at 30 s) while every TreeSA point stays at 60.5–61.8 at *all* budgets, and on surface code surgery reaches the 47.4 floor at 30 s while no TreeSA configuration gets below 48.75 at any measured budget. Orderings beyond the axis range (BFS/MCS-style treewidth, up to tc = 443) are omitted.

rators for FM to find, so the surgery advantage there is a fixed-budget effect. On the DBN instance the published (and reproduced) treewidth row wins outright (28.03, instantly, versus our 29.00)—elimination methods are the right tool for dense hyperedge-structured networks, a theme Sec. 7 develops. On nqueens_28 no stable ordering exists at these budgets: our reference's fifteen-run spread on the server spans 104.9–159.3, single Julia runs land inside that spread, and we decline to claim a direction. The Sycamore simplify-then-anneal pipeline, tuned against a faster development machine, is 0.5 *worse* than the reference at 90 s on the slower server and recovers only at 300 s—optimizer pipelines calibrate to absolute time, and cross-machine reporting must say so.

7 Application: inference workloads

Contraction ordering is not a quantum-only problem: exact probabilistic inference contracts the tensor network of a graphical model, and TensorInference.jl [21] does so literally, optimizing an einsum whose inputs are one unity tensor per variable plus one tensor per factor, at full cardinalities. The package's own cross-tool evaluation [22] shows it competitive with

	this work	Julia OMECO suite		cotengra
instance	record (90 s)	best ≤90 s	best ≤900 s	SA, 90 s
sycamore_53_20_0	**59.91**	64.49	61.35	66.03
surfacecode_d21	**47.40**	48.98	48.75	54.03
ksg	**36.96**	39.22	38.27	69.04
reg3_1000	**131.07**	—	—	165.21
dbn_13	29.00	**28.03**	**28.03**	41.15
qft_27	**29.61**	29.62	29.62	31.45

Table 2: Records versus external baselines, all measured on the same server. Julia columns: over the configuration ladder of the reference implementation (OMEinsumContractionOrders.jl: TreeSA at $\mathrm{sc}_{\mathrm{target}} \in \{20, \infty\}$ with 1/4/8 restarts, HyperND, treewidth back-ends, run through its own harness), the best row whose measured runtime fits budget B. Cotengra column: median of three 90 s single-threaded runs of the hyper-optimizer followed by subtree simulated annealing; under this protocol it is starved of the parallel trial sampling that is its main strength, so these rows are a protocol-matched reference, not a claim about cotengra at large. Bold marks the winner per instance. The dbn_13 row is won by the treewidth back-end (min-fill), instantly—see Sec. 7. reg3_1000 is not in the Julia suite. nqueens_28 is excluded: no ordering is stable at these budgets (see text).

dedicated inference engines (Merlin, libDAI, JunctionTrees) on the same networks, so its end-to-end runtime is bounded directly by the ordering quality studied here. We exported all 65 instances of the UAI-2014 MAR benchmark from the package's own artifact in exactly that form, ranked them by a 5-second hardness scan, and measured the ten hardest (three dense deep-belief networks, four pedigree-linkage networks, one CSP, one grid, plus the package's own benchmark instance Promedus_14) under the full protocol: our methods at five repetitions each, against a same-machine Julia ladder that includes the package's two defaults (the signature default GreedyMethod and the benchmark configuration TreeSA(ntrials=1, niters=5)), HyperND, the min-fill treewidth back-end, and tuned TreeSA.

Figure 5 shows every method's distance above the per-instance frontier. Three findings. *First, the shipped defaults are* 12–30 *bits off frontier* (4 000× to 10^9× in flops): GreedyMethod lands 14.6–16.6 above on the DBN family and 30.5 above on linkage_15; the benchmark TreeSA configuration is little better there (10–16 above). Whatever mechanism one prefers, an inference user who accepts defaults pays orders of magnitude for it. *Second, the frontier splits into two structural regimes.* On dense DBNs, elimination methods win outright: min-fill treewidth and HyperND reach the frontier (25.9–28.0) in milliseconds, tuned annealing trails by 5–8 bits, and our elimination-seeded annealer (Sec. 5.2) lands within 0.7 of the elimination frontier with a 10× tighter spread than the plain reference. On pedigree linkage the regimes invert: min-fill is 6–15 bits *above* the frontier, HyperND's back-end (KaHyPar) segfaults on all four instances, and annealing-family methods—ours leading on all four—hold the frontier. *Third, the regime boundary is a structure property, not a size property*: the DBN networks have 44–66 shared labels of high degree (elimination's sweet spot), while linkage networks are sparse with a thousand variables. The hard families are not a quirk of our selection: an early triage in the package's own issue tracker found 105 of the 142 UAI-2014 instances failing, and explicitly asked whether the contraction-order optimizer was leaving the DBN and pedigree families far from the treewidth optimum [23]. The reference tree decompositions posted there (Tamaki's solver) let us answer that question with an external anchor: on all four dense instances in our hard set the best elimination-method space cost equals the reference treewidth exactly (sc = 21/20/22/20 against largest bags 22/21/23/21 on DBN_12/14/16

and Grids_15), so the 12–30-bit default gap is measured against a certified-width frontier, not merely against the best order we happened to find. A portfolio that probes elimination first (milliseconds) and falls back to the annealing pipeline otherwise captures the frontier everywhere we measured; the simplification pass (Sec. 5.1) additionally absorbs the per-variable unity tensors that inference-style einsums carry by construction.

Beyond the benchmark: the excluded relational family. The same issue tracker [23] excludes one family outright—the five *relational* Markov-logic networks, "the problem scale is too large"—and no shipped test ever touches them. We exported and measured all five (Table 3); at 3 000–70 000 tensors they are up to 13× larger than anything in Sec. 6, and they take the regime split of this section to its extreme. On the two dense instances the entire annealing family—our tuned reference, the Julia TreeSA ladder, and the shipped defaults alike—lands at *exactly twice* the optimal width (largest intermediate 2^{200} versus reference treewidth 100 on relational_4; 2^{500}, the full joint, versus 250 on relational_1) at every budget up to 900 s, while elimination orderings sit exactly on the Tamaki reference width in seconds. On relational_1 our own toolchain initially produced *no order at all* within any budget: the greedy construction stage was quadratic in practice and never completed—a defect this campaign exposed and we have since fixed in the library. Surgery, instrumented on relational_4, maps its own boundary honestly: on some seeds bounded FM finds enormously cheaper cuts (36–43 bits) and surgery is the only refiner to escape the plateau (190.5 at 90 s), but the improved separators are near-cliques (clique fraction 0.41–0.44, against 0.01–0.05 on the instances of Sec. 4), and on other seeds the waist is locally minimal on every call—balanced bipartitions are structurally wrong for hub-dominated networks, whose optimal trees are deeply unbalanced elimination peelings outside the balance class surgery searches.

Answering in kind: a seed that scales. The portfolio rule above is only as good as its elimination probe, and ours froze beyond ∼4 000 labels. Diagnosing the freeze showed it was largely an implementation artifact (the quadratic greedy blocked the pipeline before the seed ran), and the cure is the standard machinery of sparse-matrix ordering: a quotient-graph elimination engine with element absorption and lazy weighted-min-degree scoring [24] orders relational_4 in 72 ms at width 101—the optimum—and relational_1 in 1.9 s at width 251. Seeded with it, our pipeline matches the elimination frontier on every relational instance (Table 3), reaches relational_5's width-10 floor at 90 s where the annealing ladder needs ∼1 000 s, and puts the first number of any method in this study on relational_1. The engine is released as a first-class `Treewidth` optimizer in the omeco library, alongside the greedy scaling fix.

8 Conclusions

We introduced global cut surgery, a contraction-order optimizer built on a structural measurement: the converged annealer's dominant contraction—the cut that carries its cost—is never a locally minimal balanced cut of the tensor graph (835 instrumented calls, zero exceptions, margins of 3–12 bits). Surgery extracts that cut, improves it globally with bounded Fiduccia–Mattheyses passes, cold-rebuilds both sides with fixed-work sub-anneals, and accepts on measured cost. Together with an exact simplification preprocessing pass, it sets records on real quantum-circuit, surface-code, and counting networks; separates fully from tuned annealing's run distribution; holds the entire high-quality wall of the same-machine time-quality Pareto front against the reference Julia implementation, its published suite, and cotengra; and its advantage grows with problem scale.

The algorithm's design is uniquely constrained by negative results: on converged instances

			ref.	annealing family (best)		elimination
instance	tensors	labels	width	either toolchain	this work + VE seed	(Julia MF)
relational_1	63 000	500	250	503.5 (sc 500)	**260.3** (sc 250)	**259.5**
relational_2	38 750	13 000	2	17.44	17.44	17.44
relational_3	3 000	1 000	7	**16.77**	16.78	16.93
relational_4	30 400	10 200	100	202.9 (sc 200)	**109.0** (sc 100)	**108.2**
relational_5	70 000	10 000	10	23.93 ($\sim 10^3$ s)	**24.03** (90 s)	**23.93** (29 s)

Table 3: The relational family, same server. "Ref. width" is the Tamaki reference treewidth from [23]. "Annealing family (best)" is the best of our tuned reference, all attempts of Secs. 5–6, and the full Julia TreeSA ladder, at budgets up to 900 s (relational_1: no method of our toolchain emitted any order before the seed of the final row existed; the value shown is Julia's default greedy after 433 s). "This work + VE seed" is the pipeline seeded by the scalable quotient-graph elimination order (median over repetitions; identical across all budgets 90–900 s). Elimination reaches the reference width exactly on every instance; the dense pair is decided by ∼94 and ∼244 bits.

the annealing frontier is certifiably width-optimal (optimum in $[53, 61.5]$ on the circuit proxy, with profile-based bounds provably exhausted), and twenty falsified mechanisms show that global information helps only as input, never as search control. The method's boundaries are mapped: expanders return the advantage at long budgets, dense inference networks belong to elimination orderings (our elimination-seeded variant comes within 0.7 bits), sparse pedigree networks belong to annealing—and shipped inference defaults sit 12–30 bits above either regime's frontier, so a milliseconds-cheap elimination probe belongs in front of every annealer.

Deliverables: the surgery pass, the simplification pass, tuned defaults, a warm-start annealing API, a scalable `Treewidth` elimination-order optimizer, a fix for a quadratic scaling defect in greedy construction, and two further bug fixes are released in the omeco library (prepared for upstreaming to OMEinsumContractionOrders.jl), together with the validator, all certificates, and the 58-attempt audit trail. Open problems: carving-width-type lower bounds; the two instances that repelled every mechanism (the 97-qubit random circuit and 28-queens, the latter unstable at practical budgets); heterogeneous bond dimensions; sliced contraction, where the interaction between slicing and surgery is unexplored; and *unbalanced* surgery—the relational family shows the balance prior excludes the elimination-like separations hub-dominated networks need, so a cut repair that searches across balance classes (or peels rather than bisects) is the natural next mechanism.

Acknowledgements

The author thanks the OMEinsumContractionOrders.jl and TensorInference.jl communities for the open benchmark suites and artifacts this study builds on.

Code and data availability The optimizer library (omeco), the validation harness, all certificates (treewidth minors, cut sets, and their verification scripts), per-attempt logs, cycle reports, the same-machine campaign data (305 + 200 + 80 + 69 + 30 scored runs, the 108-run surgery budget grid, the 324-run relational campaign, and the Julia ladders), and the UAI instance exporter are available in the project repository. Every figure regenerates from JSON data files via Typst sources shipped with the manuscript.

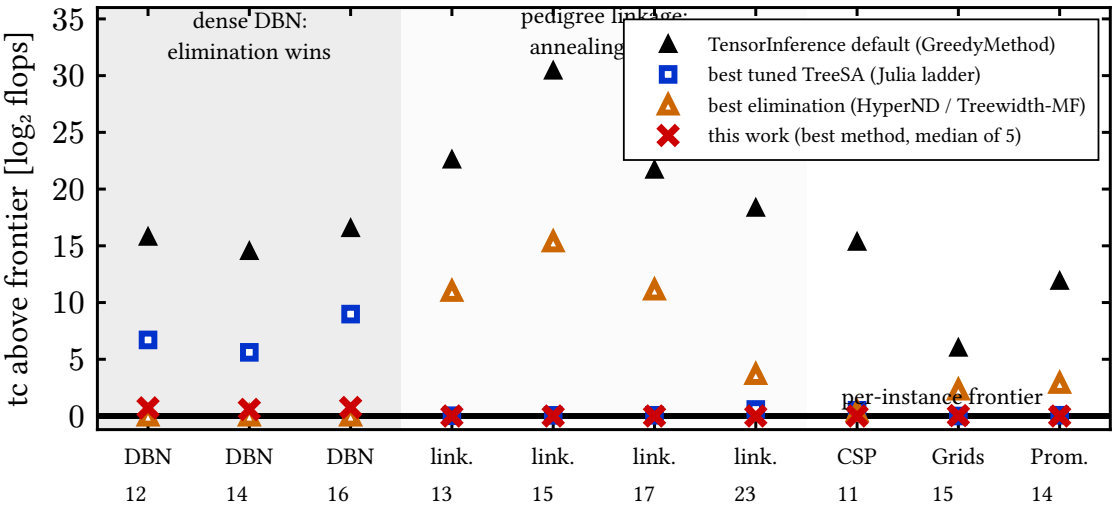

Figure 5: The ten hardest UAI-2014 MAR instances (from a 65-instance scan): each method's tc above the per-instance frontier, same machine, 90 s budgets for search methods. Filled triangles: TensorInference.jl's signature default. Squares: best tuned TreeSA from the Julia ladder. Open triangles: best elimination method (min-fill treewidth / HyperND; KaHyPar segfaults on all four linkage instances, so its rows there are min-fill only). Crosses: this work (best method, median of five). Shaded bands mark the two regimes.

Use of AI systems This research was conducted as a steered autonomous research campaign: LLM-based agents proposed hypotheses, implemented attempts in isolated worktrees, and drafted analyses, under human steering decisions documented verbatim in Appendix E and under a validation protocol in which no agent-reported number is trusted—every scored quantity is recomputed by an independent checker from artifacts, and all certified claims are verified by machine-checkable certificates independent of any agent output. The manuscript was drafted with the same agents and revised by the human author, who takes full responsibility for its content.

Funding information To be completed before submission.

A Why search control cannot win: a certified null

The case for operating on the input rather than the search is made by a null result, and we certify it. On the two closed instances the tuned annealer converges within the budget, and nothing we or an independent toolchain tried can beat it—for reasons that are provable, not anecdotal.

A.1 Thirteen mechanisms, two toolchains, one frontier

We attempted to beat the tuned reference with mechanically distinct search paradigms, implemented independently and scored under the protocol of Sec. 2: coarse subtree-transfer moves and basin hopping [17], evolutionary replica populations, exact-penalty ramps, tabu search with strategic oscillation, large-neighborhood destroy-and-repair, beam search with lookahead, Boltzmann-greedy portfolios [4, 25], recursive balanced min-cut construction [5, 26],

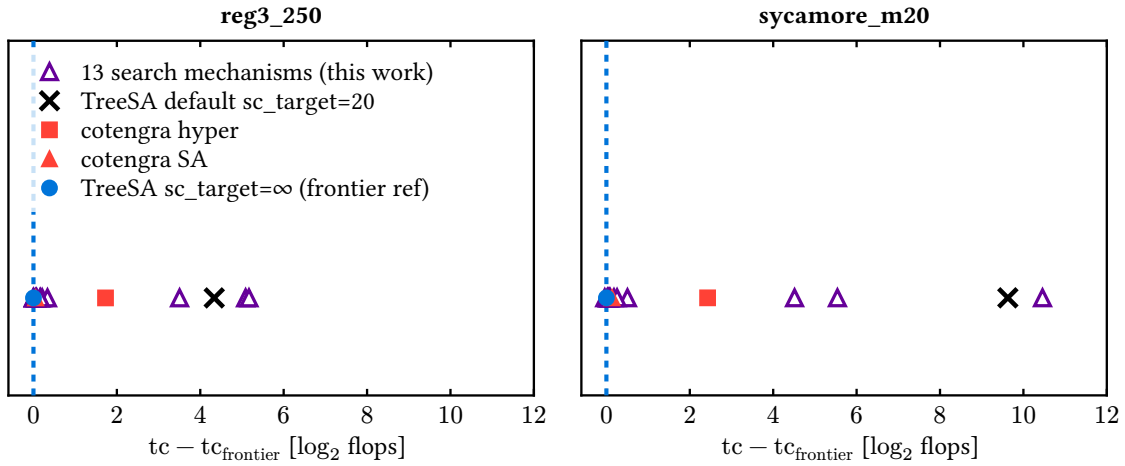

Figure 6: Distance of every scored optimizer from the frontier value (dashed line at zero). Open triangles: the thirteen search mechanisms of this work. Filled symbols: the tuned TreeSA reference ($sc_{target} = \infty$, circle), cotengra's SA refiner (triangle), cotengra's hyper-optimizer (square), and the shipped-default TreeSA (cross).

elimination-order annealing [6], order-then-tree interval dynamic programming [27], hierarchical coarsen-then-exact solves, structure-guided sweeps, and profile-aware annealing.

Figure 6 shows every scored result as its distance from the best verified value. All thirteen mechanisms land within 0.35 $\log_2$ of the tuned reference on both instances—most within noise—and none set a confirmed record. An independent toolchain reproduces the same frontier: cotengra's simulated-annealing refiner [4] ties it to within 0.05 and 0.14, while cotengra's hyper-optimization mode alone remains 1.7 and 2.4 behind at matched budget. Extending the budget tenfold moves the frontier by less than 0.1. At this scale the frontier is a property of the cost landscape, not of any search mechanism, schedule, or implementation.

A.2 Certified bounds: width-optimality and an impossibility result

Every classical lower-bound technique for contraction cost is *max-form*: it bounds the single largest contraction. The Markov–Shi correspondence [1] equates the minimal largest contraction rank with the treewidth of the line graph $L(G)$, and any balanced edge of a contraction tree yields a cut whose boundary lower-bounds the largest contraction.

On sycamore_m20 these tools give a sharp result. An explicit temporal cut severing all 53 qubit world-lines gives a balanced cut of value 53; extensive search finds nothing smaller, and the frontier trees achieve $sc = 53$ exactly. The frontier is therefore *width-optimal*, and $tw(L(G)) = 52$. Combined with the frontier value, we obtain the certified localization

$$tc_{min}(\text{sycamore\_m20}) \in [\,53,\ 61.544\,]. \tag{A.1}$$

On reg3_250 the analogous certification is obstructed by the known hardness of certifying high treewidth on expanders: the best machine-verified certificate (a 32-vertex minor of $L(G)$, checked by two independent exact solvers) gives $tw(L(G)) \geq 18$ against a constructive upper bound of 34.

The width bound has a ceiling: tc is a log-sum (1) over $\sim|V|$ contractions, and the frontier's excess over the width bound—8.5 on sycamore_m20, close to the maximal possible $\log_2 561 = 9.1$—is invisible to any bound on a single contraction. Closing the interval (A.1) from below requires *sum-form* bounds. For any tree, each internal node v with leaf set S_v satisfies $cost_v \geq |\partial S_v|$, the boundary of S_v in G. Writing $b(k)$ for the isoperimetric profile of G (the minimal boundary over all k-vertex subsets), we prove:

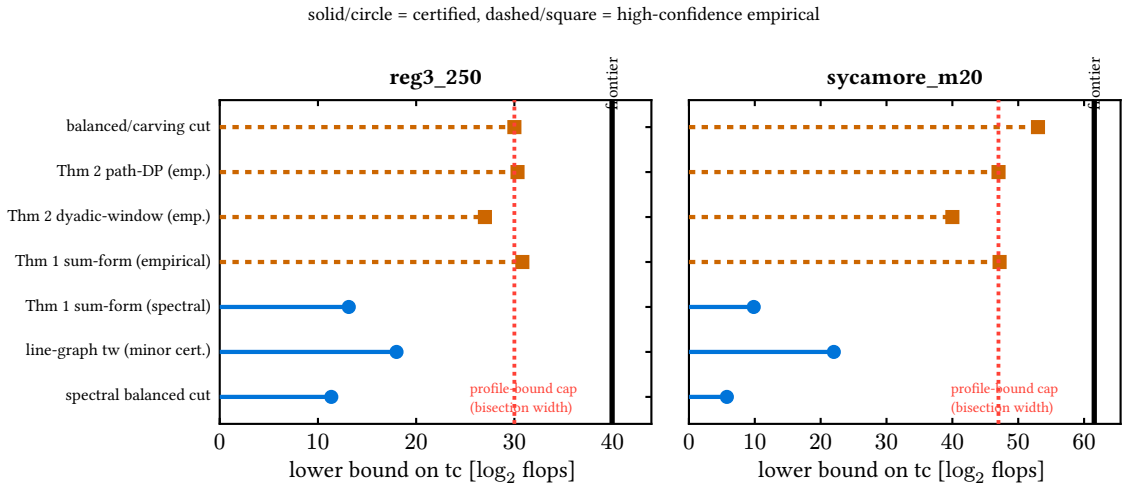

Figure 7: Lower-bound ladder for both closed instances. Solid stems with circles are certified (machine-checkable certificates); dashed stems with squares are high-confidence empirical values. Every profile-based bound (Theorems 1 and 2) is capped at the bisection width (dotted line, Observation 1); only the balanced/carving cut exceeds it. The solid vertical line is the optimizer frontier.

Theorem 1 (Sum-form profile bound). *For every contraction tree,* $\mathrm{tc} \geq \log_2 F(n)$ *where* F *obeys the recursion* $F(1) = 0$, $F(k) = 2^{b(k)} + \min_{1 \leq j \leq k/2} [F(j) + F(k-j)]$.

Theorem 2 (Dyadic-window bound). *Descending from the root always into the larger child visits, for every dyadic window* $W_j = (n/2^{j+1}, n/2^j]$, *at least one internal node whose leaf-set size lies in* W_j*; these nodes are distinct, hence* $\mathrm{tc} \geq \log_2 \sum_j 2^{\min_{k \in W_j} b(k)}$.

Both theorems are validated computationally (exhaustively at small n, and on 3000 random trees). Both fail to beat the max-form cap (Fig. 7): on sycamore_m20 they reach 47.2 and 47.0 against the carving value 53. The reason is structural:

Observation 1 (Profile bounds are capped at the bisection width). *Any lower bound computed from the isoperimetric profile* $b(\cdot)$ *alone is at most* $\max_k b(k) + \log_2 \log_2 n$, *and* $\max_k b(k)$ *is the bisection width—strictly below the balanced-cut (carving) value whenever the profile dips off-center. On sycamore_m20 it does: we exhibit a triply-verified 141-tensor subset with boundary exactly 40, against temporal-slab boundaries of 53 (Fig. 8).*

The 47 → 53 gap lives in jointly-nested-cut structure that per-size minima discard; the 53 → 61.5 gap is the log-sum residual. A complementary measurement (Fig. 9) shows why search cannot exploit the residual either: reducing the largest contraction by one unit systematically multiplies the number of contractions just below the peak, so total flops behave as if conserved under profile reshaping. This certified null fixes the design space for any challenger: the gains must come from changing what is searched, not how.

B What does not work: falsified alternatives

Twenty further mechanisms were implemented, measured, and falsified under the same protocol, and their taxonomy (Table 4) is what makes the design of Sec. 5 non-arbitrary: *every mechanism that injected global information as search control failed; every mechanism that injected it as input won.* Construction-time global structure (separators, spectral cuts, boundary combs)

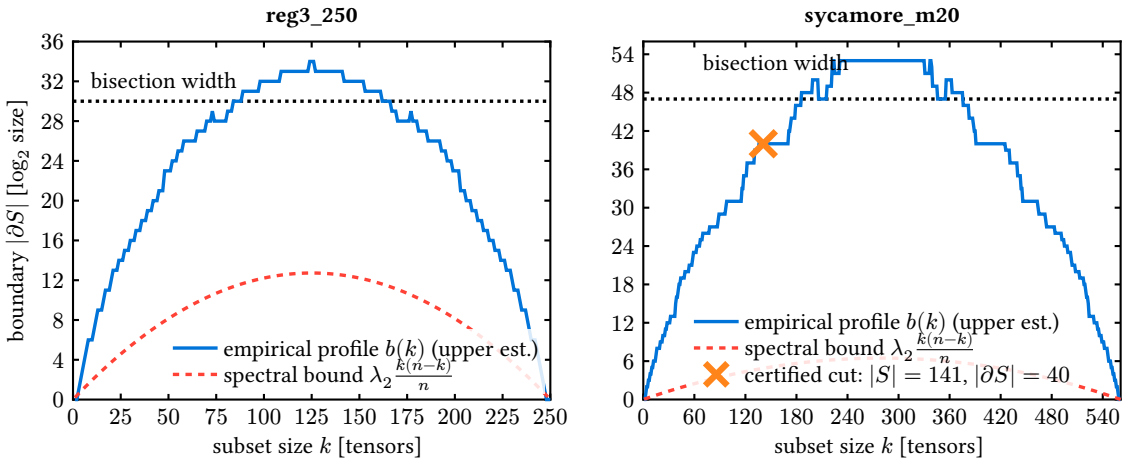

Figure 8: Isoperimetric profiles. The empirical profile $b(k)$ (solid) dips well below the bisection width (dotted) at off-center sizes; the certified marker shows the 141-tensor sycamore_m20 subset with boundary exactly 40. This dip is what caps all profile-based bounds (Observation 1).

loses to the greedy portfolio it seeds; search-control globality (targeting, tempering, blocking, acceptance relaxation) breaks the annealer's schedule invariants; exhaustive enumeration hits certified combinatorial walls (window-exact optimality of the incumbent frontier; block counts exploding past 10^5 within bits of the minimum boundary—with an outer-product counterexample showing the connectivity-restricted variant is not even correct); and population policies drain a small recombination reservoir (about ten productive recombinations) regardless of diversity management. Global cut surgery is, to our knowledge, the unique surviving design point: it consumes global information (a better cut) as an *input* to a rebuild, leaving the annealer's control loop untouched.

C Supplementary figures

D Methodology: a research loop that distrusts itself

The results above were produced by an autonomous research loop—56 optimizer attempts over ten cycles, each in an isolated worktree with a pre-registered hypothesis, scored by a sandboxed validator—and the loop's central design lesson is that its most important outputs were *retractions* (Fig. 14). Three failure modes were caught by the harness rather than by luck:

(i) Schedule exploitation. The first gate (a speed-versus-baseline bar) was passed by budget hygiene alone—anytime writes and early termination—with no algorithmic content. The fix was structural: fold the exploit into the reference, making schedule-only candidates score zero by construction, and keep the exploit as a permanent negative control.

(ii) Mis-tuned-baseline gains. Eight mechanisms then "beat" the references by up to 800×—until a strengthened reference (the one-line $\mathrm{sc}_{\mathrm{target}}$ fix of Sec. 3) reclaimed the entire gain. What survived was the tuning discovery itself, and the leaderboard practice of ratcheting references to absorb every discovered improvement.

(iii) Instrument bias. A failed attempt's diagnosis revealed that the independent scorer silently rejected contraction trees deeper than ∼475 (a recursion limit), biasing the admissible space against exactly the deep, MPS-like trees that suit circuit instances. The scorer was fixed,

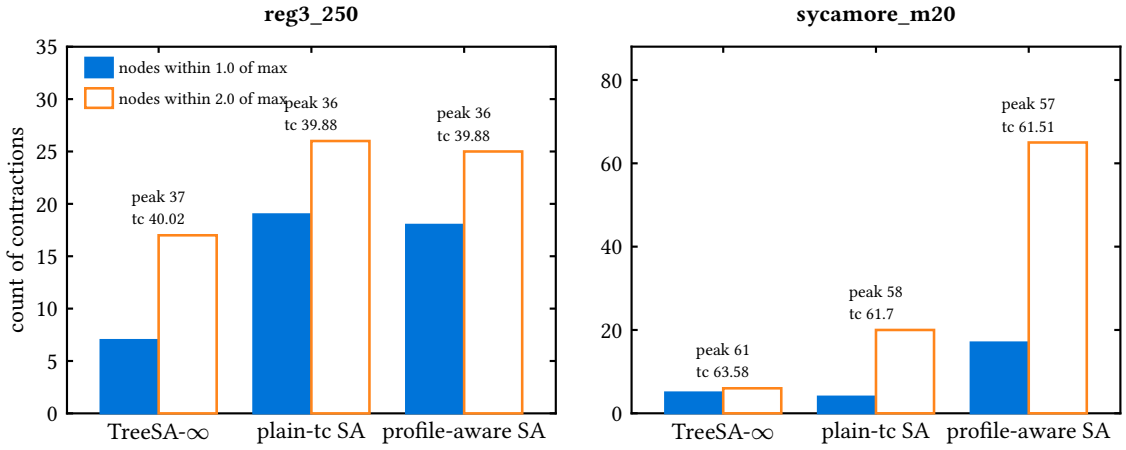

Figure 9: Cost-profile conservation at the frontier. Bars count contractions within 1.0 (solid) and 2.0 (outline) of the largest; labels give each tree's peak cost and tc. Trees that lower the peak by one unit acquire several times more near-peak contractions at essentially unchanged tc.

its negative-control suite re-passed, and the affected hypothesis re-tested.

As the campaign scaled, the harness grew with it: parallel scoring lanes with per-child CPU accounting, median-of-three scoring above 1000 tensors, early abort of hopeless confirmation runs, and harness-clocked anytime curves that make time-to-frontier claims backdate-proof. The pattern that generalizes: score only through a hardened independent validator; maintain executable negative controls, and when a new exploit is found, patch the harness and add the exploit to the control suite; ratchet references so yesterday's wins are tomorrow's baselines; and report distributions wherever single runs are noisy. Under this regime, every claim in this paper is either machine-certified, or labeled high-confidence with its evidence, or was retracted before publication—and the audit trail (per-attempt logs, certificates, and cycle reports) ships with the code.

E How the algorithm was found: steered autonomous research

The algorithm of this paper was produced by an autonomous research loop (machine-proposed hypotheses, isolated implementations, hardened scoring) that was *steered* by a small number of human decisions. (For LLM-driven *heuristic evolution* on this same problem—automated search over code rather than a steered hypothesis loop—see [10].) Because the loop retained every artifact, the causal path from the opening question to the algorithm can be reconstructed exactly. Figure 15 shows that path left to right: the top row quotes the steering prompts, the middle row shows only the attempts that carried results forward (most of the 56 attempts served as controls, falsifications, or dead branches feeding Appendices B and D), and each arrow states why the transition happened.

Three properties of this trajectory are worth stating explicitly. *First, every pivot was forced by a measurement, not chosen freely.* The move from tc-optimization to the anytime axis happened because thirteen mechanisms and a second toolchain had converged on one frontier that was then certified locally optimal three independent ways; the move to scale happened because time-to-frontier proved heavy-tailed at the claim threshold; the move to global-information mechanisms happened because every search-control variant had failed with a structural cause while every input-level variant had won. *Second, the human contributions were few but load-*

family / mechanism (attempt)	measured cause of failure
Construction proxies	
BFS/planar separators (037)	seeds lose to greedy portfolio even with separator injection
spectral bisection (041)	power iteration cannot converge (gap $\sim 1/n^2$); a perfect bisection is still dominated
boundary-MPS combs (036/044)	cutwidth 90–126 lower-bounds any comb, far above balanced-tree records
mass randomized greedy (040)	quality ceiling 15–208 bits above frontier at 1.2 builds/s
Search-control globality	
elimination-order annealing (045)	flat landscape: 30 improvements per 2.5M moves; representation floor $\sim 2\times$ above tree space
congestion-softmax targeting (042)	collapses onto the dominant node on peaky profiles; 40–80% dead traversal
cache-blocked regional SA (043)	pointer trees have no memory contiguity; blocking starves global moves
replica-exchange tempering (048)	swap acceptance $\to$ 0% at scale (energy gaps grow with n)
late-acceptance hill climbing (051)	never cools on plateau-dominated landscapes; schedules are load-bearing
MCTS over prefixes (050)	reaches depth 3 of $\sim$380 at 8–25 rollouts/s
Exhaustive enumeration	
window-exact splice (033)	frontier trees are window-exact optimal (≤ 16 leaves); the dominant window spans a constant fraction
Tamaki-style block DP (056)	feasible low-boundary blocks explode past 10^5–10^6 within bits of $\min_W$; outer-product counterexample falsifies the connectivity-restricted variant
Population policy	
path-relinking / consensus / diversity pools (046/049/055)	basin complementarity drains after $\sim$10 recombinations regardless of pool policy

Table 4: The falsified-mechanism taxonomy. Each row is an implemented, scored, and diagnosed attempt; attempt numbers index the public audit trail. The four families fail for four distinct measured reasons, and none of the failures is a tuning accident.

bearing: the demand for a strengthened reference (which produced the tuning result of Sec. 3), the switch to real-provenance instances (which unlocked the simplification pass—the same mechanism had been correctly measured as useless on the pre-simplified proxy two cycles earlier), the question "can global information help the local search escape?" (which became global cut surgery), and the directive to demonstrate the methods on inference workloads (which produced the two-regime result of Sec. 7). *Third, retests are cheap when provenance changes*: attempt 013's null on the synthetic proxy and attempt 039's records on the real circuit are the same mechanism measured on different inputs, and only the pre-registered instance provenance made the contradiction interpretable rather than embarrassing.

References

[1] I. L. Markov and Y. Shi, *Simulating quantum computation by contracting tensor networks*, SIAM Journal on Computing **38**(3), 963 (2008), doi:10.1137/050644756, quant-ph/0511069.

[2] Y. Liu, Y. Chen, C. Guo, J. Song, X. Shi, L. Gan, W. Wu, W. Wu, H. Fu, X. Liu, D. Chen, Z. Zhao *et al.*, *Verifying quantum advantage experiments with multiple amplitude tensor network contraction.*, Physical review letters **132 3**, 030601 (2022).

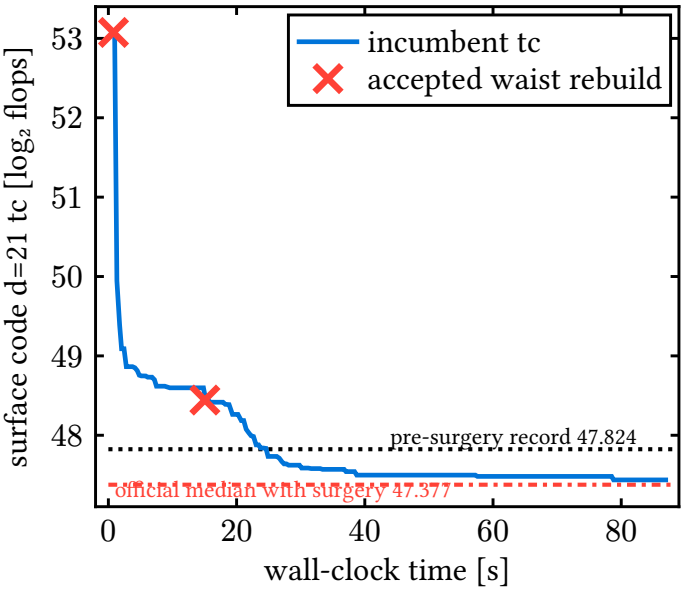

Figure 10: A representative surgery run on surface code d=21: incumbent tc (solid), accepted waist rebuilds (crosses), the pre-surgery record (dotted) and the official with-surgery median (dash-dotted). Each accepted rebuild moves the waist; surgery repeats at the new waist until no improving cut remains.

[3] G. Kalachev, P. Panteleev and M.-H. Yung, *Multi-tensor contraction for xeb verification of quantum circuits* (2021), https://arxiv.org/abs/2108.05665.

[4] J. Gray and S. Kourtis, *Hyper-optimized tensor network contraction*, Quantum **5**, 410 (2020).

[5] C. Staudt, M. Blacher, J. Klaus, F. Lippmann and J. Giesen, *Improved cut strategy for tensor network contraction orders*, In *22nd International Symposium on Experimental Algorithms (SEA 2024)*, vol. 301 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 27:1–27:19, doi:10.4230/LIPIcs.SEA.2024.27 (2024).

[6] E. Dumitrescu, A. L. Fisher, T. Goodrich, T. Humble, B. D. Sullivan and A. Wright, *Benchmarking treewidth as a practical component of tensor network simulations*, PLoS ONE **13** (2018).

[7] H. Tamaki, *Positive-instance driven dynamic programming for treewidth*, Journal of Combinatorial Optimization **37**, 1283 (2017).

[8] R. Pfeifer, J. Haegeman and F. Verstraete, *Faster identification of optimal contraction sequences for tensor networks.*, Physical review. E, Statistical, nonlinear, and soft matter physics **90 3**, 033315 (2013).

[9] E. A. Meirom, H. Maron, S. Mannor and G. Chechik, *Optimizing tensor network contraction using reinforcement learning*, In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, vol. 162 of *Proceedings of Machine Learning Research*, pp. 15278–15292 (2022), https://arxiv.org/abs/2204.09052.

[10] F. Hoppe, M. Röhrig-Zöllner and P. Knechtges, *Algorithmic algorithm development with llms: A case study on llm usage for contraction order optimization in tensor networks* (2026), https://arxiv.org/abs/2606.01975.

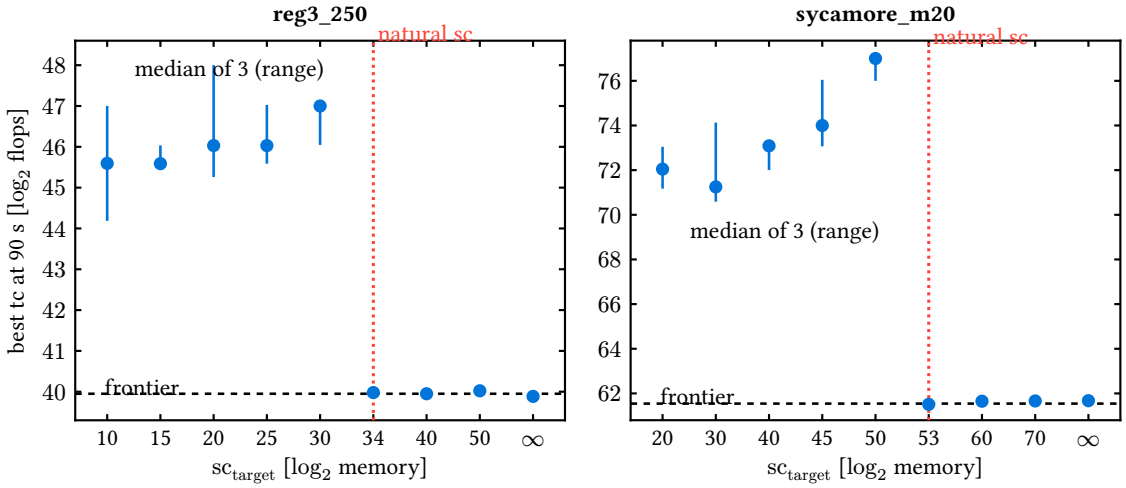

Figure 11: Best tc reached in 90 s of single-threaded annealing versus the memory target $\mathrm{sc}_{\mathrm{target}}$ (median of three runs; whiskers span the range). The dashed line marks the converged frontier of Appendix A; the dotted vertical line marks the natural space cost of optimum-quality trees. The transition is a cliff at the natural value: every smaller target is comparably harmful, and every target at or above it, including $\mathrm{sc}_{\mathrm{target}} = \infty$, reaches the frontier. The shipped default is $\mathrm{sc}_{\mathrm{target}} = 20$.

[11] J. Xu, H. Zhang, L. Liang, L. Deng, Y. Xie and G. Li, *Np-hardness of tensor network contraction ordering* (2023), https://arxiv.org/abs/2310.06140.

[12] T. Ibaraki and T. Kameda, *On the optimal nesting order for computing n-relational joins*, ACM Transactions on Database Systems (TODS) **9**, 482 (1984).

[13] C.-C. Lam, P. Sadayappan and R. Wenger, *On optimizing a class of multi-dimensional loops with reductions for parallel execution*, Parallel Process. Lett. **7**, 157 (1997).

[14] M. Stoian, R. M. Milbradt and C. Mendl, *On the optimal linear contraction order of tree tensor networks, and beyond*, SIAM J. Sci. Comput. **46**, 647 (2022).

[15] J.-G. Liu, X. Gao and R. Samuelson, *Omeinsumcontractionorders: A julia package for tensor network contraction order optimization*, J. Open Source Softw. **11**, 9886 (2026).

[16] C. M. Fiduccia and R. M. Mattheyses, *A linear-time heuristic for improving network partitions*, 19th Design Automation Conference pp. 175–181 (1982).

[17] M. Geiger, Q. Huang and C. B. Mendl, *Optimizing tensor network partitioning using simulated annealing*, ArXiv:2507.20667 (2025).

[18] R. D. Guerrero, *Bond-dimension scaling of a local-refinement advantage over hyperoptimized tensor-network contraction on sycamore-like topologies* (2026), https://arxiv.org/abs/2604.25532.

[19] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. B. Costa, Y.-L. L. Fang, Y. Gao, J. Guan, J. A. Gunnels, A. Haidar, A. Hehn, M. Hohnerbach *et al.*, *cuquantum sdk: A high-performance library for accelerating quantum science*, 2023 IEEE International Conference on Quantum Computing and Engineering (QCE) **01**, 1050 (2023).

[20] D. Smith and J. Gray, *opt_einsum - a python package for optimizing contraction order for einsum-like expressions*, J. Open Source Softw. **3**, 753 (2018).

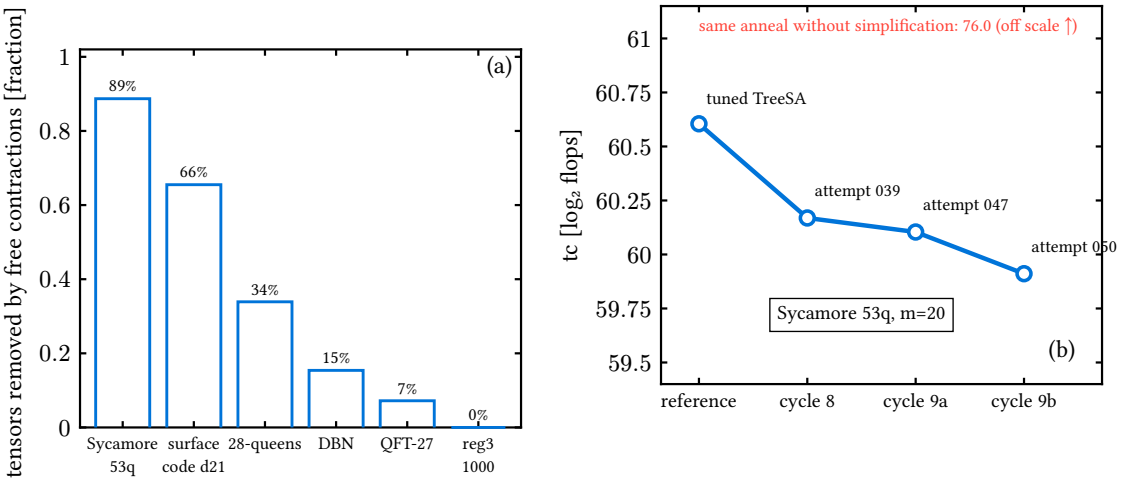

Figure 12: The preprocessing pass. (a) Fraction of tensors removed by the exact rank-non-increasing pass; the random-regular control is untouched at 0%. (b) The Sycamore 53q record march: the tuned reference (leftmost) and three successive simplify-then-anneal attempts. The same anneal without simplification reaches 76.0 (off scale): preprocessing, not search, carries this improvement.

[21] M. Roa-Villescas and J.-G. Liu, *TensorInference: A Julia package for tensor-based probabilistic inference*, Journal of Open Source Software **8**(90), 5700 (2023), doi:10.21105/joss.05700.

[22] M. Roa-Villescas and J.-G. Liu, *TensorInferenceBenchmarks.jl: performance evaluation of TensorInference.jl against Merlin, libDAI, and JunctionTrees on the UAI 2014 networks*, https://github.com/TensorBFS/TensorInferenceBenchmarks.jl (2023).

[23] M. Roa-Villescas, *Inference errors or fails for 105 of the 142 UAI 2014 inference competition problems*, TensorInference.jl issue #15, https://github.com/TensorBFS/TensorInference.jl/issues/15 (2022).

[24] P. R. Amestoy, T. A. Davis and I. S. Duff, *An approximate minimum degree ordering algorithm*, SIAM J. Matrix Anal. Appl. **17**(4), 886 (1996), doi:10.1137/S0895479894278952.

[25] S. Orgler and M. Blacher, *Optimizing tensor contraction paths: A greedy algorithm approach with improved cost functions* (2024), https://arxiv.org/abs/2405.09644.

[26] B. Strasser, *Computing tree decompositions with flowcutter: Pace 2017 submission* (2017), https://arxiv.org/abs/1709.08949.

[27] C. Ibrahim, D. Lykov, Z. He, Y. Alexeev and I. Safro, *Constructing optimal contraction trees for tensor network quantum circuit simulation*, 2022 IEEE High Performance Extreme Computing Conference (HPEC) pp. 1–8 (2022).

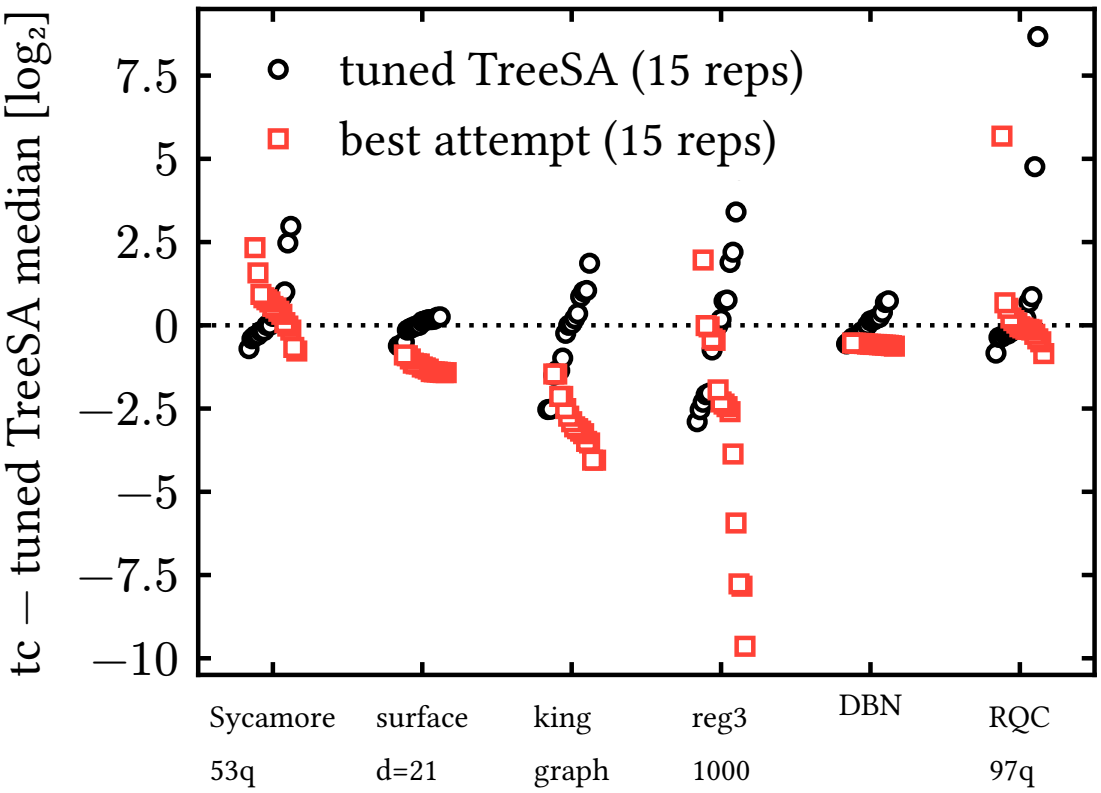

Figure 13: Run distributions of the same-machine campaign: all 15 repetitions per instance at 90 s, centered on the tuned reference's median. Circles are the tuned reference, squares the best method per instance.

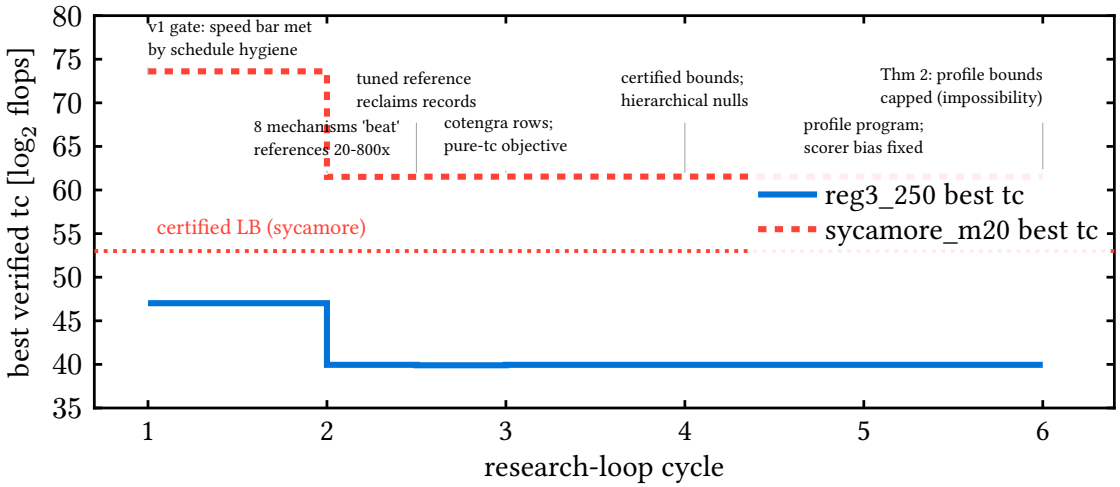

Figure 14: Best verified tc over the research-loop cycles. Apparent progress in cycle 2 is reclaimed by the strengthened reference (cycle 2.5); thereafter the closed-instance frontier is flat while the certified picture grows, and cycles 7–10 move the record board on the large structured instances instead. Annotations mark the three instrument-caught failure modes.

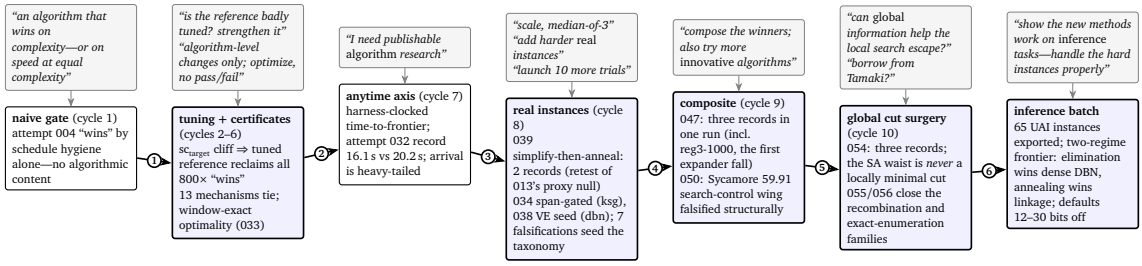

Figure 15: The research roadmap, left to right. Top row (grey): the human steering prompts, essentially verbatim. Bottom row: the result-carrying attempts only; most of the 56 attempts are controls and falsifications that feed Table 4 and Appendix D. Each numbered arrow is a transition forced by a measurement: (1) the gate was passed without an algorithm, so the gate was hardened and references ratcheted; (2) tc saturated and was certified locally optimal, so the objective axis changed to arrival time; (3) speed gains sat at the confirmation noise floor while scale budgets were non-converged, so the regime changed; (4) the winning mechanisms proved complementary, so they were composed while exploration continued; (5) every win injected global information as *input* while every search-control variant failed, so global information was aimed directly at the local search's blind spot—the dominant cut; (6) the mechanism portfolio was complete, so it was pointed at a second application domain to test transfer.